

“University Laboratory Management Server”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

A V S N R AKHILA	2520030022
GUTHA SUSHMA SREE	2520030098
MANASVI NILESH GHAG	2520030335

Under the Esteemed Guidance of

K. Hema

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“University Laboratory Management Server”** is a record of Bonafide work of **A V S N R AKHILA - 2520030022, GUTHA SUSHMA SREE - 2520030098, MANASVI NILESH GHAG - 2520030335** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A V S N R AKHILA	2520030022
GUTHA SUSHMA SREE	2520030098
MANASVI NILESH GHAG	2520030335

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“University Laboratory Management Server”** is a bonafide work done and submitted by **A V S N R AKHILA – 2520030022, GUTHA SUSHMA SREE – 2520030098, MANASVI NILESH GHAG – 2520030335,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **K. Hema**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

A V S N R AKHILA 2520030022

GUTHA SUSHMA SREE 2520030098

MANASVI NILESH GHAG 2520030335

Abstract

A university computer laboratory has a limited number of resources such as computers, printers, file servers, projectors, scanners and headsets, but many students need them at the same time. When these requests are handled manually, the same resource may be given to two students, some students wait for a long time, and no proper record of usage is kept. To solve this, our project focuses on the design and development of a University Laboratory Management Server. Our core aim was to implement a centralised server in the C language on Linux (Ubuntu) that can handle multiple student requests concurrently while controlling access to the limited laboratory resources.

The server is built around the main concepts of operating systems. Students send requests through TCP sockets from a client program or a web page. The main thread accepts the requests and places them in a shared request queue, and three worker threads created with POSIX threads take the requests from the queue and process them. Mutexes protect the shared queue and the allocation table from race conditions, and one counting semaphore for each resource type makes sure that a resource is given only when an instance is free. If no instance is free, the student waits until one is released.

A separate timer thread checks the usage time and releases every resource automatically when its time is over. The administrator controls the server with signals: SIGUSR1 shows the pending requests, SIGUSR2 shows the resource status, and SIGINT or SIGTERM performs a controlled shutdown. Every activity is recorded with date and time in a log file. A simple web dashboard made with HTML, CSS and JavaScript is served by the same C server to show the available resources, student status and current allocations. In summary, this project delivers a complete and working application that demonstrates threads, synchronisation, inter-process communication, signals and file handling in a real-life situation.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–6
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		7–13
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		14–16
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		17–20
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		21–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	Implementation		30–50
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		51–59
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		60–63
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		64–66
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

A university computer laboratory is one of the busiest shared facilities on a campus. In a single lab session, a large number of students use the systems at the same time. They log in to computers, print their records, store programs on the file server, use the projector for seminars, scan documents and use headsets for language and communication labs. All these facilities are limited in number, but the students who want to use them are many. Because of this, the lab needs a proper way to decide which student gets which resource and for how long.

In most laboratories this work is still done manually. A lab in-charge keeps a register, notes down the student ID and the system number, and tells the students when a system or a printer becomes free. This method works when the number of students is small, but it becomes slow and confusing when many students ask for resources at the same moment. Two students may be told to use the same system, a student may keep a resource for a long time without anybody noticing, and there is no clear record of who used what and when.

This situation is very similar to the main job of an operating system. An operating system also has a small number of resources such as the CPU, memory, files and devices, and a large number of processes that want to use them at the same time. The operating system solves this by using concepts such as processes, threads, scheduling queues, mutual exclusion, semaphores, inter-process communication and signals. So a laboratory management system is a very good real-life example for understanding and applying these operating system concepts.

With this idea, we developed the University Laboratory Management Server. It is a centralised server program written in the C language on Linux (Ubuntu). Students send their requests to the server from a client program or from a web page. The server accepts many requests at the same time, keeps them in a shared request queue, and uses worker threads to process them. Mutexes protect the shared data,

counting semaphores control the limited laboratory resources, a timer thread releases a resource automatically when the requested time is over, signals are used by the administrator to check the status and shut the server down safely, and every activity is stored in a log file.

1.2 Problem Statement

The laboratory needs a centralised management system that can handle multiple student requests concurrently while controlling access to limited resources. In detail, the problem given to us is to build a university laboratory management server in C that:

- accepts multiple client requests simultaneously,
- uses worker threads to process student requests,
- maintains a shared request queue,
- uses mutexes to protect shared data structures,
- uses semaphores to control access to limited laboratory resources,
- maintains student requests and resource allocation information,
- uses signals for server administration and controlled shutdown,
- monitors active processes and threads,
- maintains server activity logs, and
- detects and reports resource usage and availability.

The main difficulty in this problem is concurrency. When several students send requests at the same instant, the server must not lose any request, must not give the same resource to more students than the number available, and must not corrupt its own data. If two threads change the same variable at the same time, the result becomes unpredictable. This is called a race condition. The server therefore has to be designed in such a way that all shared data is accessed safely and all limited resources are counted correctly at every moment.

1.3 Scope of the Project

The scope of the project covers the complete server side of a laboratory resource management system and a simple user interface for demonstrating it. The following points come under the scope of this project:

- A TCP server that runs on Ubuntu and listens on port 8080 for student requests.
- Six types of laboratory resources: computers, printers, file servers, projectors, scanners and headsets.
- A pool of three worker threads that take requests from a shared circular queue and process them.
- A timer thread that checks the usage time every second and releases expired resources automatically.
- An allocation table that stores the student ID, resource, duration, start time and expected end time of every allocation.
- A resource dashboard, a list of all allocations and a student-wise status report.
- Signal handling for viewing pending requests, viewing resource status and controlled shutdown.
- An activity log file (server.log) with the date and time of every event in Indian Standard Time.
- A terminal client program and a browser-based dashboard made with HTML, CSS and JavaScript, both of which talk to the same C server.

The project is meant for demonstrating operating system concepts on a single machine. Features such as student login with passwords, permanent storage of data in a database, and control of real hardware devices are outside the present scope and are listed as future work in the last chapter.

1.4 Objectives of the Project

The objectives of the project are taken directly from the problem statement. Each objective is connected with one operating system concept that we studied in the course.

1. To accept multiple client requests simultaneously.
2. To use worker threads to process student requests.
3. To maintain a shared request queue.
4. To use mutexes to protect shared data structures.
5. To use semaphores to control limited resources.
6. To maintain student requests and resource allocation information.
7. To use signals for server administration and controlled shutdown.
8. To monitor threads and resource usage.
9. To maintain server activity logs.

Table 1.1: Objectives and the related OS concept

Objective	Related OS concept
Accept multiple client requests	Inter-process communication using TCP sockets
Process requests using worker threads	Multithreading with POSIX threads
Maintain a shared request queue	Bounded buffer (producer-consumer problem)
Protect shared data	Mutual exclusion using mutex locks
Control limited resources	Counting semaphores
Maintain allocation information	Shared data structures and critical sections
Server administration and shutdown	Signals and signal handlers
Monitor threads and resources	Process and thread monitoring
Maintain activity logs	File input and output

1.5 Importance of the Application

The application is important from two points of view. From the point of view of a laboratory, it gives a single place where all resource requests are received and answered. Students do not have to wait for the lab in-charge to check a register. A

request is answered within a moment, the resource is given only if it is really free, and it comes back to the pool automatically when the time is over. The dashboard shows at any time how many computers, printers and other resources are free, and the log file gives a complete history of the activities in the lab.

From the point of view of learning, the application is important because it puts almost all the major topics of the operating systems course into one working program. Reading about a semaphore in a textbook is different from seeing a fourth student wait because only three resources exist, and then seeing that student get the resource the moment another student releases it. In the same way, the need for a mutex becomes clear only when several threads try to update the same queue. This project allowed us to see these ideas working in front of us.

- It avoids double allocation of the same resource.
- It reduces manual work for the lab in-charge.
- It gives fair, first-come-first-served service to students through the request queue.
- It releases resources automatically, so no resource stays blocked after the usage time.
- It keeps a permanent record of all activities for later checking.
- It can be monitored and stopped safely by the administrator without losing data.

1.6 Target Users / Audience

The system is designed for the people who use or manage a university computer laboratory. The main users are given below.

Table 1.2: Target users of the system

User	How they use the system
Students	Request a resource for a given time, view the resource dashboard and check their own status.
Lab in-charge / Administrator	Starts and stops the server, sends signals to see pending requests and resource status, and reads the log file.

Faculty	View the allocations during a lab session and check the usage of laboratory resources.
OS course learners	Study the program as a working example of threads, mutexes, semaphores, sockets and signals.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

This section explains the operating system concepts that are used in the project. For each concept we first give the idea in simple words and then say where it is used in our server.

2.1.1 Process

A process is a program in execution. When we type `./lab_server` in the terminal, the operating system creates a process for our server. The process has its own address space, which contains the code, the global variables, the heap and the stack. It also has a process ID (PID), which is used by the administrator to send signals to the server. Every run of the client program (`./lab_client`) is another separate process. So in our project the server and each client are different processes, and they communicate with each other through sockets.

2.1.2 Thread

A thread is a small unit of execution inside a process. All the threads of a process share the same code, global variables and open files, but each thread has its own stack and program counter. Creating a thread is cheaper than creating a process, and threads can share data directly without any special communication mechanism. For this reason threads are the natural choice for a server that has to do many similar jobs at the same time.

Our server uses five threads in total: one main thread that accepts client connections, three worker threads that process the requests, and one timer thread that releases expired resources. The threads are created with the POSIX thread library (pthreads).

2.1.3 Concurrency and Race Condition

Concurrency means that several activities are in progress during the same period of time. When concurrent threads read and write the same data without any control,

the final result depends on the exact order in which the threads run. This situation is called a race condition. For example, if two worker threads read the value 'available computers = 1' at the same time and both decrease it, both will believe they got the last computer. The part of the code where shared data is accessed is called the critical section, and only one thread should be inside it at a time.

In our server the shared data are the request queue, the array of available resources and the allocation table. All of these are accessed by more than one thread, so each of them is a critical section that must be protected.

2.1.4 Mutual Exclusion and Mutex

Mutual exclusion means that if one thread is executing in its critical section, no other thread is allowed to execute in the same critical section. A mutex (mutual exclusion lock) is the simplest tool for this. A thread locks the mutex before entering the critical section and unlocks it after leaving. If another thread tries to lock the same mutex in between, it is made to wait.

We use two mutexes. The mutex `queue_mutex` protects the request queue, and `resource_mutex` protects the resource counters and the allocation table. Using two separate mutexes means that a thread that is updating the allocation table does not block another thread that only wants to add a request to the queue.

2.1.5 Semaphore

A semaphore is an integer variable that is accessed only through two atomic operations, traditionally called wait (P) and signal (V). The wait operation decreases the value, and if the value is already zero, the calling thread is blocked. The signal operation increases the value and wakes up one waiting thread, if any. A binary semaphore takes only the values 0 and 1 and works like a lock. A counting semaphore can take any non-negative value and is used to control access to a resource that has several identical instances.

Laboratory resources are a perfect example for counting semaphores. If the lab has ten computers, the computer semaphore starts with the value 10. Each time a

computer is given to a student, `sem_wait()` reduces the value by one. When the value becomes zero, the next student who asks for a computer has to wait. When a computer is released, `sem_post()` increases the value and the waiting student continues. Our server has one counting semaphore for each of the six resource types.

2.1.6 Condition Variable

A condition variable allows a thread to sleep until some condition becomes true. It is always used together with a mutex. In our server the worker threads have nothing to do when the request queue is empty. Instead of checking the queue again and again and wasting CPU time (busy waiting), a worker calls `pthread_cond_wait()` and sleeps. When the main thread adds a new request, it calls `pthread_cond_signal()` and one sleeping worker wakes up. We use two condition variables, `queue_not_empty` and `queue_not_full`.

2.1.7 Producer-Consumer Problem

The producer-consumer (bounded buffer) problem is a classical synchronisation problem. A producer puts items into a buffer of fixed size and a consumer takes them out. The producer must wait when the buffer is full and the consumer must wait when it is empty. Our request queue is exactly this problem: the main thread is the producer of requests, the three worker threads are the consumers, and the circular queue of size 100 is the bounded buffer.

2.1.8 Inter-Process Communication and Sockets

Processes do not share memory by default, so they need a mechanism to exchange data. This is called inter-process communication (IPC). Pipes, message queues, shared memory and sockets are the common IPC mechanisms in Linux. A socket is an end point of communication and it works both between processes on the same machine and between different machines on a network. We use TCP stream sockets, which give a reliable, connection-oriented byte stream. The server creates a socket, binds it to port 8080, listens on it and accepts connections. Each client

creates a socket and connects to the server.

2.1.9 Signals

A signal is a software interrupt sent to a process to inform it about an event. Signals are asynchronous, which means they can arrive at any time. A process can install a function called a signal handler, and the kernel calls this function when the signal arrives. In our server SIGUSR1 is used to print the number of pending requests, SIGUSR2 is used to print the resource status, and SIGINT (Ctrl+C) and SIGTERM are used for a controlled shutdown.

2.1.10 File Management

The operating system provides files for storing data permanently. A program opens a file, reads or writes it and closes it. Our server uses file I/O in two places: it appends every activity to the file server.log, and it reads the HTML, CSS and JavaScript files of the dashboard from the disk and sends them to the browser.

2.1.11 Deadlock

A deadlock is a situation in which a set of threads wait for each other for ever. It can occur only when four conditions hold together: mutual exclusion, hold and wait, no pre-emption and circular wait. While designing the server we took care to avoid deadlock. A worker thread never holds resource_mutex while it is blocked on a semaphore, the two mutexes are never locked in opposite orders by different threads, and every lock has a matching unlock on all paths of the code.

Table 2.1: OS concepts and where they are used in the project

Concept	Where it is used in our project
TCP Sockets (IPC)	Communication between client and server
POSIX Threads	Worker threads and timer thread
Mutex	Protects request queue and allocation table
Condition Variable	Wakes a worker when a request arrives
Counting Semaphore	Controls limited laboratory resources
Signals	Server status and controlled shutdown
File I/O	Activity log in server.log
Dynamic Memory	Worker IDs and web page data

2.2 Existing Methods / Algorithms

Before deciding our design we studied the existing ways in which a laboratory is managed and the existing ways in which a server can handle many clients.

2.2.1 Manual Register Method

In the manual method the lab in-charge writes the student ID, system number and time in a register. This method needs no software, but it is slow, depends fully on one person, and cannot handle many requests at the same time. There is no automatic release of resources and no quick way to know how many resources are free.

2.2.2 Iterative (Single-Threaded) Server

The simplest server handles one client at a time. It accepts a connection, finishes the complete work for that client and only then accepts the next one. This is easy to write and has no race conditions, but if one request takes a long time, for example when a student is waiting for a busy resource, every other student is blocked. So this method does not satisfy the requirement of handling multiple requests simultaneously.

2.2.3 Process-per-Client Server

In this method the server calls `fork()` for every new connection and the child process serves the client. Many clients can be served together, but creating a process is costly, and since every child has its own copy of the memory, the children cannot directly share the queue and the allocation table. Extra IPC such as shared memory would be needed to keep the resource counts common for all.

2.2.4 Thread-per-Client Server

Here the server creates a new thread for each connection. Threads share memory, so the data can be shared easily. But if a hundred students connect at once, a hundred threads are created, and the time spent in creating, switching and destroying threads becomes large. The number of threads is not under control.

2.2.5 Synchronisation Methods

For protecting shared data, the older methods are disabling interrupts, busy-waiting lock variables and Peterson's solution. These either work only on a single processor or waste CPU time in a loop. Modern systems use blocking primitives such as mutexes, semaphores and monitors, in which a waiting thread is put to sleep by the operating system and woken up only when it can proceed.

Table 2.2: Comparison of server models

Method	Handles many clients	Shares data easily	Overhead
Manual register	No	No	High human effort
Iterative server	No	Yes	Low
Process per client	Yes	No	High
Thread per client	Yes	Yes	Grows with clients
Thread pool with queue (ours)	Yes	Yes	Low and fixed

2.3 Proposed Approach

We propose a thread pool server with a shared request queue. The server creates a fixed number of worker threads once, at the time of starting. The main thread only accepts connections, reads each request and puts it into a circular queue. The worker threads sleep on a condition variable and wake up when a request is available. A worker takes one request from the queue, processes it, sends the reply to the student and goes back for the next request.

The limited laboratory resources are controlled by counting semaphores, one for each type of resource. The resource counters and the allocation table are protected by a mutex. A separate timer thread wakes up every second, checks the expected end time of every allocation and releases the resources whose time is over. The administrator controls the server using signals, and all activities are written to a log file.

For the user interface we do not create a second server. The same C server also

understands simple HTTP requests from a web browser, so the web dashboard and the terminal client are two different faces of the same server. Whatever a student requests from the web page passes through the same allocation logic in C.

2.4 Advantages of the Proposed Approach

- **Concurrency:** three requests can be processed at the same time, and a student waiting for a busy resource does not stop the others.
- **Controlled overhead:** the number of threads is fixed, so the server does not slow down when many students connect together.
- **Fairness:** requests are served in the order in which they arrive because the queue is first-in-first-out.
- **Correctness:** mutexes remove race conditions, and semaphores make sure that the number of allocations never exceeds the number of resources.
- **No busy waiting:** idle workers sleep on a condition variable and use no CPU time.
- **Automatic release:** the timer thread returns a resource to the pool when the usage time is over, so resources are not blocked for ever.
- **Easy administration:** the status can be seen at any time with a signal, without stopping the server.
- **Safe shutdown:** on SIGINT or SIGTERM the server stops accepting requests, joins all threads and destroys the semaphores before exiting.
- **Traceability:** the log file gives the complete history with date and time.
- **Single server for both interfaces:** the terminal client and the web dashboard always show the same data.

CHAPTER -3 SYSTEM REQUIREMENTS

This chapter lists the hardware and software needed to develop and run the University Laboratory Management Server. The project is light and does not need any special hardware. Any laptop or desktop that can run Ubuntu is enough.

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 64-bit processor	Quad-core processor (Intel i5 / Ryzen 5 or above)
RAM	4 GB	8 GB or more
Storage	2 GB free space	10 GB free space
Display	1366 x 768	1920 x 1080
Network	Loopback interface (localhost)	LAN / Wi-Fi for access from other systems
Input devices	Keyboard and mouse	Keyboard and mouse

A multi-core processor is recommended because the benefit of multithreading is seen clearly only when the worker threads can really run in parallel on different cores. On a single-core machine the threads still work correctly, but they are executed one after another by time sharing.

3.2 Software Requirements

Table 3.2: Software requirements

Requirement	Details
Operating System	Ubuntu (Linux)
Language	C
Compiler	GCC
Libraries	pthread, semaphore, socket, signal
Frontend	HTML, CSS, JavaScript
Web browser	Firefox or Google Chrome
Editor	nano / any text editor
Version control	Git and GitHub
Files	lab_server.c, lab_client.c, server.log

All the libraries used in the project are part of the standard GNU C library (glibc)

that comes with Ubuntu. No external library has to be downloaded. The frontend uses only plain HTML, CSS and JavaScript, so no web framework, package manager or separate web server is required.

3.3 Operating System Environment – Ubuntu / WSL

The project is developed and tested on Ubuntu, which is a popular Linux distribution. Linux is chosen because the project depends on POSIX features such as pthreads, POSIX semaphores, BSD sockets and signals like SIGUSR1 and SIGUSR2. These features are available directly in Linux. The same program can also be run on Windows through the Windows Subsystem for Linux (WSL), which provides a full Ubuntu environment inside Windows.

The Ubuntu terminal is used for everything in this project: writing the code with the nano editor, compiling with GCC, running the server and the clients in different terminal windows, sending signals with the kill command and reading the log file. Several terminals are opened at the same time. One terminal runs the server, the other terminals act as different students, and one more terminal is used by the administrator to send signals.

- Ubuntu 22.04 LTS or later (native installation, virtual machine or WSL 2).
- Bash shell and the standard GNU core utilities.
- The build-essential package, which installs GCC, make and the C library headers.
- Time zone support (tzdata), since the server prints all times in Indian Standard Time.

The build-essential package can be installed with the commands `sudo apt update` and `sudo apt install build-essential`. After this, the command `gcc --version` should display the version of the compiler.

3.4 Development Tools and Compilers

Table 3.3: Development tools

Tool	Purpose in the project
GCC	Compiles lab_server.c and lab_client.c into executable files
nano	Terminal text editor used to write and modify the source files
make / Makefile	Stores the compile commands so that the project can be built with one command
curl	Sends test requests to the server from the terminal
ps, pgrep, top	Show the process ID and the threads of the running server
kill	Sends SIGUSR1, SIGUSR2 and SIGTERM to the server
cat, tail	Display the contents of server.log
Web browser developer tools	Check the requests made by the dashboard and debug JavaScript
Git / GitHub	Store and share the project files

The compiler is used with the `-pthread` option. This option tells GCC to compile and link the program with the POSIX thread library. Without it the calls to `pthread_create()` and the semaphore functions are not linked properly.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The complete server and the terminal client are written in the C language. C is the language in which Linux itself is written, and all the system calls of the operating system are directly available as C functions. This makes C the most suitable language for a systems programming project. With C we can create sockets, start threads, lock mutexes, wait on semaphores and install signal handlers without any extra layer in between.

The following features of C are used in the project:

- Structures (struct) to represent a Request and an Allocation.
- Arrays to implement the circular request queue and the allocation table.
- Pointers and function pointers, for example the thread function passed to `pthread_create()` and the handler passed to `signal()`.
- Preprocessor constants such as `PORT`, `QUEUE_SIZE` and `WORKER_COUNT`, so that the configuration can be changed in one place.
- String functions such as `snprintf()`, `strtok()`, `strncmp()` and `strcat()` to build replies and to parse requests.
- Dynamic memory allocation with `malloc()` and `free()`.
- Standard file functions `fopen()`, `fprintf()` and `fclose()` for logging.

C++ is not required for this project. The program is kept in plain C so that it stays close to the operating system interfaces that we studied in the course.

4.2 GCC Compiler

GCC (GNU Compiler Collection) is the standard compiler on Linux. It converts the C source file into an executable file in four stages: preprocessing, compilation, assembly and linking. In the linking stage the object code of our program is joined with the C library and the thread library.

Table 4.1: GCC options used

Option	Meaning
-o name	Gives the name of the output executable file
-pthread	Compiles and links with the POSIX thread library
-Wall	Shows all common warnings, which helps to find mistakes early
-g	Adds debugging information for use with a debugger

The server is compiled with the command `gcc lab_server.c -o lab_server -pthread` and the client with `gcc lab_client.c -o lab_client`. A Makefile is kept in the project folder so that both programs can be built together.

4.3 Linux System Calls

A system call is the way in which a user program asks the kernel to do some work on its behalf, such as creating a socket or writing to a file. The C library provides a wrapper function for each system call. The important system calls and library functions used in the project are grouped below.

Table 4.2: System calls and library functions used

Group	Calls	Use in the project
Socket	<code>socket()</code> , <code>setsockopt()</code> , <code>bind()</code> , <code>listen()</code> , <code>accept()</code> , <code>connect()</code>	Create the server and client end points and make connections
Data transfer	<code>send()</code> , <code>recv()</code>	Exchange requests and replies
Closing	<code>close()</code> , <code>shutdown()</code>	Close a client connection and stop the listening socket
Signals	<code>signal()</code> , <code>kill (command)</code>	Install handlers and send signals to the server
Time	<code>time()</code> , <code>localtime()</code> , <code>strftime()</code> , <code>difftime()</code> , <code>sleep()</code>	Time stamps, usage time and the one-second timer
Files	<code>fopen()</code> , <code>fprintf()</code> , <code>fread()</code> , <code>fseek()</code> , <code>ftell()</code> , <code>fclose()</code>	Write the log and read the web pages
Memory	<code>malloc()</code> , <code>free()</code> , <code>memset()</code>	Allocate and clear memory
Environment	<code>setenv()</code> , <code>tzset()</code>	Set the time zone to Asia/Kolkata

4.4 POSIX Threads / Synchronization

POSIX threads, usually called pthreads, is the standard thread library on Linux and other UNIX-like systems. It is declared in the header file `pthread.h`. POSIX

semaphores are declared in semaphore.h. These two headers give us all the tools needed for multithreading and synchronisation.

Table 4.3: Thread and synchronisation functions used

Function	Purpose
pthread_create()	Creates a new thread that starts running the given function
pthread_join()	Waits for a thread to finish; used during shutdown
pthread_mutex_lock()	Locks a mutex before entering a critical section
pthread_mutex_unlock()	Unlocks the mutex after leaving the critical section
pthread_cond_wait()	Releases the mutex and sleeps until the condition is signalled
pthread_cond_signal()	Wakes up one thread waiting on the condition
pthread_cond_broadcast()	Wakes up all waiting threads; used at shutdown
sem_init()	Creates a semaphore with an initial count
sem_wait()	Decreases the count; blocks if the count is zero
sem_post()	Increases the count and wakes a waiting thread
sem_destroy()	Destroys the semaphore when the server stops

The mutexes and condition variables are created with the static initialisers PTHREAD_MUTEX_INITIALIZER and PTHREAD_COND_INITIALIZER, so they are ready to use as soon as the program starts. The semaphores are initialised in main() with the number of instances of each resource.

4.5 Linux Commands and Utilities

A number of Linux commands were used while developing, running and demonstrating the project.

Table 4.4: Linux commands used

Command	Use
mkdir, cd, ls	Create the project folder and move between folders
nano file.c	Open a source file in the editor
gcc	Compile the programs
./lab_server	Start the server
./lab_client	Start a student client
ps -ef grep lab_server	Find the server process
pgrep lab_server	Print the process ID of the server
ps -T -p <pid>	List the threads of the server process

kill -SIGUSR1 <pid>	Ask the server to print the pending requests
kill -SIGUSR2 <pid>	Ask the server to print the resource status
kill -SIGTERM <pid> or Ctrl+C	Shut the server down safely
cat server.log, tail -f server.log	Read the activity log
curl http://localhost:8080/api/dashboard	Test the server from the terminal
git add, git commit, git push	Upload the project to GitHub

4.6 Other Tools / Libraries Used

- **HTML:** gives the structure of the web dashboard, with sections for Dashboard, Request Resource, Student Status, Allocated Resources and Server Response.
- **CSS:** gives the layout, colours and the light and dark themes of the dashboard.
- **JavaScript (Fetch API):** sends requests from the browser to the C server and shows the replies on the page without reloading it.
- **HTTP:** the simple text protocol that the browser uses to talk to the server. Our C server reads the first line of the HTTP request and sends back a proper HTTP response.
- **Web browser:** displays the dashboard at the address <http://localhost:8080>.
- **Git and GitHub:** used to keep the project files safe and to submit them for review.

No database, web framework or third-party library is used. Everything that the browser needs is served by our own C program.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows the client-server architecture. There is one server process and any number of client processes. The clients are of two kinds: the terminal client program and the web browser. Both connect to the server on TCP port 8080. Inside the server there are three layers: the connection layer (main thread), the processing layer (request queue and worker threads) and the resource layer (semaphores, resource counters and allocation table). The timer thread, the signal handler and the logger work beside these layers.

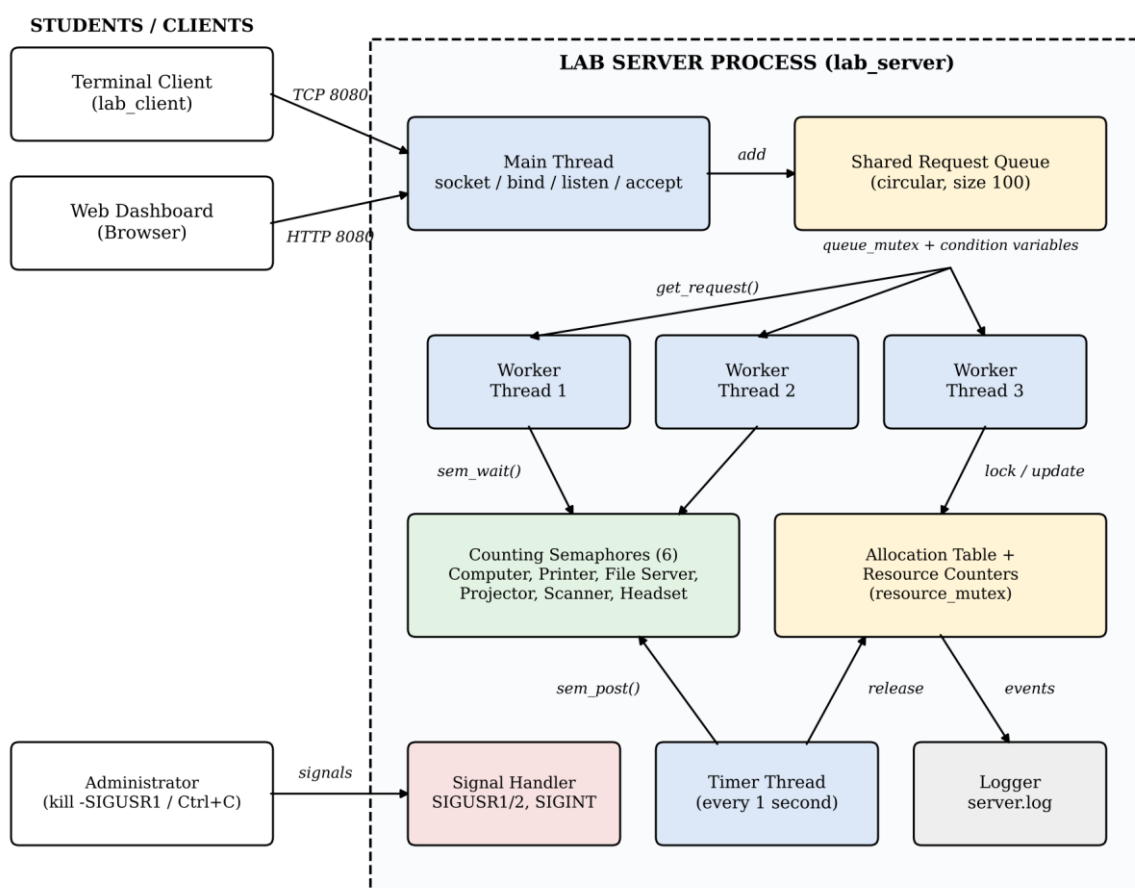


Figure 5.1: Overall architecture of the University Laboratory Management Server

The main thread is the only thread that touches the listening socket. It accepts a connection, reads the request and places it in the request queue. It never waits for a resource, so it is always free to accept the next student. The worker threads are the only threads that wait on semaphores. If all the instances of a resource are in use,

only that worker waits, and the other workers continue to serve other students. The timer thread never talks to clients. Its only job is to return expired resources to the pool.

Table 5.1: Components of the architecture

Component	Responsibility
Terminal client	Shows the menu, takes the student ID and choice, sends the request and prints the reply
Web dashboard	Shows resource cards, request form, status and allocations in the browser
Main thread	Creates the socket, accepts connections and adds requests to the queue
Request queue	Circular buffer of 100 requests shared by the main thread and the workers
Worker threads (3)	Take requests from the queue, allocate resources and send replies
Semaphores (6)	Count the free instances of each resource type
Allocation table	Stores who is using which resource and until when
Timer thread	Releases resources whose usage time is over
Signal handler	Handles SIGUSR1, SIGUSR2, SIGINT and SIGTERM
Logger	Appends every event to server.log with the time

5.2 System Workflow

The working of the system from the moment a student sends a request to the moment the resource is released is given in the following steps.

1. Student sends a request from the client / web page.
2. Server receives the request through a socket (port 8080).
3. Request is added to the shared request queue.
4. A worker thread takes the request from the queue.
5. Mutex locks the shared data, semaphore checks if resource is free.
6. Resource is allocated to the student.
7. Timer thread releases the resource after the given time.
8. Every activity is written in server.log.

In the first step the student enters the student ID, selects a resource and gives the

usage time in seconds. The client joins these values into one line of text in the format `student_id|request_type|resource|duration|message` and sends it to the server. In the second step the main thread accepts the connection and reads this line. It splits the line at the '|' characters and fills a Request structure.

In the third step the main thread locks `queue_mutex`, copies the request to the rear of the circular queue, signals the condition variable `queue_not_empty` and unlocks the mutex. In the fourth step one of the sleeping worker threads wakes up, removes the request from the front of the queue and starts processing it.

In the fifth step the worker first checks that the resource number and duration are valid and that the same student does not already hold the same resource. Then it calls `sem_wait()` on the semaphore of that resource. If an instance is free the call returns immediately; otherwise the worker sleeps until an instance is released. In the sixth step the worker reduces the available count, adds a new entry to the allocation table with the start time and the expected end time, and sends the reply 'RESOURCE ALLOCATED' to the student.

In the seventh step the timer thread, which wakes up every second, finds that the current time has crossed the expected end time of the allocation. It marks the entry as released, increases the available count and calls `sem_post()`, which wakes up any worker waiting for that resource. In the eighth step, which actually happens along with every other step, the event is written to the log file.

5.3 Flowchart / Data Flow

The flowchart in Figure 5.2 shows the logic followed by a worker thread for one resource request. Figure 5.3 shows how the data moves between the student, the server and the data stores.

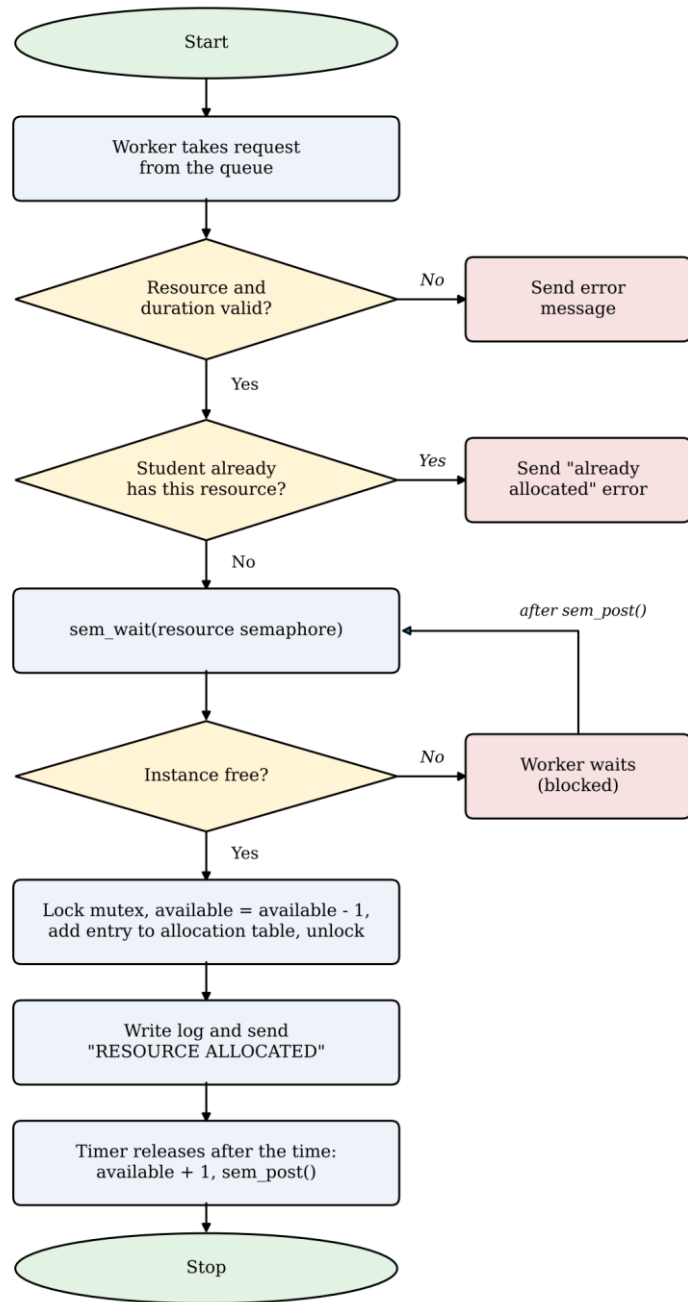


Figure 5.2: Flowchart for processing a resource request

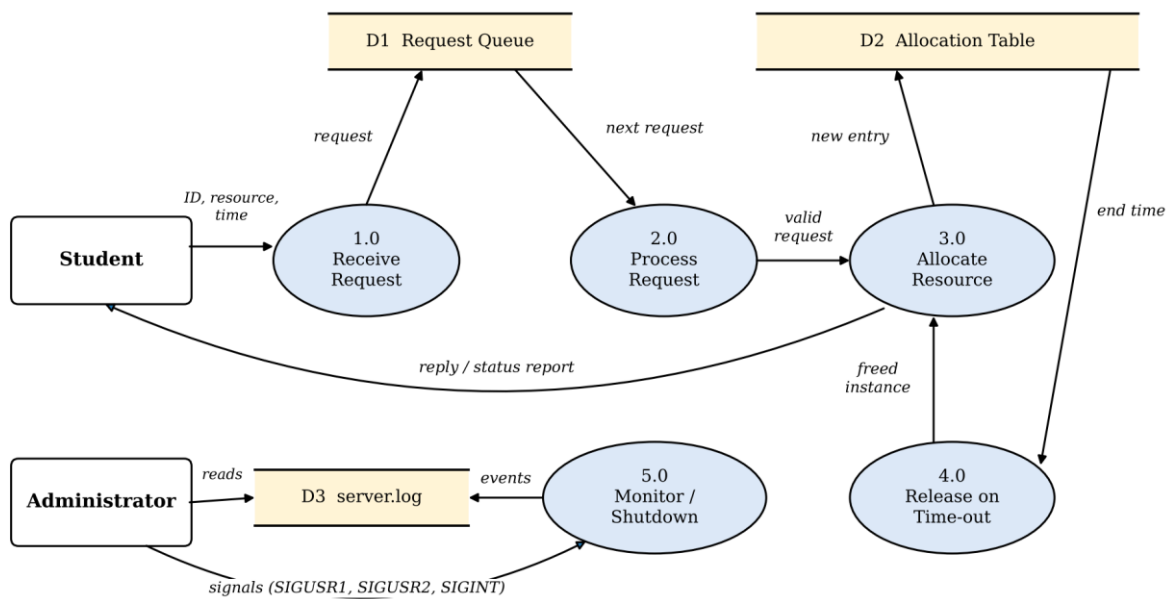


Figure 5.3: Data flow diagram of the system

In the data flow diagram the student is the external entity who gives the request details and receives the reply. The administrator is the second external entity who sends signals and reads the log. The request queue, the allocation table and the log file are the three data stores. The request moves from the student to the queue, from the queue to a worker, and the result of the worker updates the allocation table and the log file. The timer process reads the allocation table and updates it when it releases a resource.

5.4 Module Description

The server is divided into the following modules. Each module is a group of related functions in lab_server.c.

Table 5.2: Modules and their functions

Module	Main functions	Description
Connection module	main()	Creates the socket, binds, listens, accepts clients and parses requests
Request queue module	add_request(), get_request()	Implements the circular queue with a mutex and two condition variables

Worker module	<code>worker_thread()</code>	Takes requests and calls the correct function according to the request type
Resource allocation module	<code>process_resource_request()</code> , <code>add_allocation()</code> , <code>has_duplicate_allocation()</code>	Checks, allocates and records a resource
Timer module	<code>timer_thread()</code> , <code>check_expired_allocations()</code> , <code>release_allocation()</code>	Releases expired resources automatically
Reporting module	<code>show_resource_status()</code> , <code>show_all_allocations()</code> , <code>show_student_status()</code>	Prepares the dashboard, the allocation list and the student status
Signal module	<code>handle_signal()</code>	Administration and controlled shutdown
Logging module	<code>write_log()</code> , <code>get_indian_time()</code>	Writes time-stamped entries to <code>server.log</code>
Frontend module	HTTP handling in <code>main()</code> , <code>index.html</code> , <code>style.css</code> , <code>script.js</code>	Serves the web dashboard from the same server

5.4.1 Data Structures

Two structures carry most of the information in the server. The Request structure represents one request sent by a student, and the Allocation structure represents one resource currently given to a student.

Table 5.3: Fields of the Request structure

Field	Type	Meaning
<code>client_socket</code>	int	Socket on which the reply has to be sent
<code>student_id</code>	int	ID of the student who sent the request
<code>request_type</code>	int	1-6 for a resource, 7 dashboard, 8 all allocations, 9 my status
<code>resource</code>	int	Resource number (1 Computer ... 6 Headset)
<code>duration</code>	int	Requested usage time in seconds
<code>message</code>	char[256]	Short text sent by the client

Table 5.4: Fields of the Allocation structure

Field	Type	Meaning
<code>student_id</code>	int	Student who holds the resource
<code>resource</code>	int	Resource number
<code>duration</code>	int	Requested usage time in seconds
<code>start_time</code>	time_t	Time at which the resource was given

expected_end_time	time_t	start_time + duration
allocated	int	1 while in use, 0 after release

Table 5.5: Laboratory resources and their semaphores

Resource	Total Available	Semaphore
Computers	10	computer_sem
Printers	5	printer_sem
File Servers	5	fileserver_sem
Projectors	5	projector_sem
Scanners	5	scanner_sem
Headsets	5	headset_sem

5.5 Input and Output Design

5.5.1 Input Design

The inputs are kept as few and as simple as possible. A student has to give only three values: the student ID, the choice of resource or report, and the usage time in seconds. In the terminal client these are typed at the prompts. In the web dashboard they are entered in a small form with a text box for the student ID, a drop-down list for the resource and a number box for the duration.

Menu shown by the terminal client

```
=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====

Enter Student ID: 101

----- MENU -----
1. Request Computer
2. Request Printer
3. Request File Server
4. Request Projector
5. Request Scanner
6. Request Headset
7. View Resource Dashboard
8. View All Allocated Resources
9. View My Resource Status
-----

Enter your choice: 1

Enter required usage time (in seconds): 60
```

The inputs are validated at two places. The client rejects an invalid menu choice and a duration that is zero or negative before contacting the server. The server

checks the resource number and the duration again, and also checks whether the same student already holds the same resource. In that case it replies with an error message and does not allocate the resource a second time.

Table 5.6: Request message format

Position	Field	Example
1	Student ID	101
2	Request type	1
3	Resource	1
4	Duration in seconds	60
5	Message	Computer

For the example in the table, the client sends the line 101|1|1|60|Computer to the server.

5.5.2 Output Design

The outputs of the system are text reports that are easy to read both in the terminal and in the browser. There are four kinds of output: the allocation reply, the resource dashboard, the list of all allocations and the status of one student. The server also prints its own progress messages in the server terminal and writes them to the log file.

Reply sent to the student after a successful allocation

```
=====
                        RESOURCE ALLOCATED
=====
Student ID   : 101
Resource     : Computer
Duration     : 60 seconds
Start Time   : 10:15 07-10-2026 IST
Expected End : 10:16 07-10-2026 IST
=====
The resource will be automatically
released after 60 seconds.
=====
```

Resource dashboard

```
=====
                        LAB RESOURCE DASHBOARD
=====
Computers    : 9/10 available
Printers     : 5/5 available
File Servers : 5/5 available
Projectors   : 5/5 available
Scanners     : 5/5 available
```

```
Headsets      : 5/5 available
```

```
=====
```

The times and dates in the two listings above are only examples to show the format. The actual outputs obtained during execution are given in Chapter 7.

In the web dashboard the same information is shown in a more visual way. The dashboard page shows one card for each resource with the number available out of the total. The Request Resource page has the input form. The Student Status and Allocated Resources pages show the reports, and the Server Response page shows the exact text returned by the C server. A navigation bar at the top is used to move between the pages, and a button changes between the light and dark themes.

CHAPTER -6 IMPLEMENTATION

This chapter explains how the design given in the previous chapter is implemented in C. The server is written in a single source file, `lab_server.c`, and the terminal client in `lab_client.c`. The web dashboard consists of three files, `index.html`, `style.css` and `script.js`, kept in the folder `lab_frontend`. The important parts of the code are given as listings and are explained one by one.

6.1 Module-wise Implementation

Table 6.1: Project files

File	Contents
<code>lab_server.c</code>	Main server program: sockets, 3 worker threads, timer thread, mutexes, semaphores, request queue, signals, resource allocation, logging, automatic release
<code>lab_client.c</code>	Student client program: menu, resource requests, dashboard, allocated resources, student status
<code>lab_frontend/index.html</code>	Structure of the web dashboard
<code>lab_frontend/style.css</code>	Styles and themes of the web dashboard
<code>lab_frontend/script.js</code>	Sends requests from the browser to the C server and updates the page
<code>Makefile</code>	Compile commands for the project
<code>server.log</code>	Activity log generated by the server

6.1.1 Connection Module

The connection module is the first part of `main()`. It creates a TCP socket with `socket(AF_INET, SOCK_STREAM, 0)`, sets the `SO_REUSEADDR` option so that the port can be used again immediately after a restart, binds the socket to port 8080 on all interfaces and starts listening. After that it enters a loop in which it accepts one connection at a time, reads the request text with `recv()` and converts it into a Request structure.

6.1.2 Request Queue Module

The queue is a global array of 100 Request structures with three integers: `queue_front`, `queue_rear` and `queue_count`. The function `add_request()` inserts at the rear and `get_request()` removes from the front. Both positions move in a

circular manner using the modulo operator. All access to these variables is done with `queue_mutex` locked.

6.1.3 Worker Module

Each worker runs the function `worker_thread()`. It is an endless loop that calls `get_request()`, looks at the request type and calls the function for that type. Request types 1 to 6 are resource requests, 7 is the dashboard, 8 is the list of all allocations and 9 is the status of the student. After the reply is sent the worker closes the client socket.

6.1.4 Resource Allocation Module

The function `process_resource_request()` contains the main logic of the project. It validates the request, checks for a duplicate allocation, waits on the semaphore of the resource, updates the available count and records the allocation. The functions `get_resource_name()` and `get_resource_semaphore()` convert a resource number into its name and its semaphore using a switch statement.

6.1.5 Timer Module

The timer thread calls `sleep(1)` and then `check_expired_allocations()` in a loop. This function locks `resource_mutex`, goes through the allocation table and releases every entry whose expected end time has passed. For every released entry it calls `sem_post()` on the semaphore of that resource.

6.1.6 Reporting Module

Three functions prepare the reports. `show_resource_status()` prints the available and total count of every resource. `show_all_allocations()` lists every active allocation with requested time, used time, start time and expected end time. `show_student_status()` shows the same details, together with the remaining time, for one student. All three lock `resource_mutex` while reading the shared data so that they get a consistent picture.

6.1.7 Signal and Logging Modules

The function `handle_signal()` is registered for four signals. The function

`write_log()` opens `server.log` in append mode, writes the current time in Indian Standard Time followed by the message, and closes the file. Opening and closing the file for every entry makes sure that the entry reaches the disk even if the server is stopped suddenly.

6.2 Algorithm / Pseudocode

6.2.1 Main Thread

Algorithm 1: Main thread

```
BEGIN
  set time zone to Asia/Kolkata
  initialise six semaphores with the resource totals
  register handlers for SIGUSR1, SIGUSR2, SIGINT, SIGTERM
  create WORKER_COUNT worker threads
  create the timer thread
  create socket, bind to PORT, listen
  WHILE server_running DO
    client = accept(connection)
    text = receive(client)
    request = parse(text)           // id | type | resource | duration |
message
    add_request(request)
  END WHILE
  wake all waiting threads
  join worker threads and timer thread
  destroy semaphores
  write "server stopped" to log
END
```

6.2.2 Request Queue

Algorithm 2: Adding and removing a request

```
add_request(request):
  lock(queue_mutex)
  WHILE queue is full DO wait(queue_not_full, queue_mutex)
  queue[rear] = request
  rear = (rear + 1) mod QUEUE_SIZE
  count = count + 1
  signal(queue_not_empty)
  unlock(queue_mutex)

get_request():
  lock(queue_mutex)
  WHILE queue is empty DO wait(queue_not_empty, queue_mutex)
  request = queue[front]
  front = (front + 1) mod QUEUE_SIZE
  count = count - 1
  signal(queue_not_full)
  unlock(queue_mutex)
  RETURN request
```

6.2.3 Worker Thread and Resource Allocation

Algorithm 3: Worker thread

```
worker_thread(id):
    WHILE server_running DO
        request = get_request()
        IF request.type is 1..6 THEN process_resource_request(request)
        ELSE IF request.type = 7 THEN show_resource_status()
        ELSE IF request.type = 8 THEN show_all_allocations()
        ELSE IF request.type = 9 THEN show_student_status(request.student_id)
        ELSE send "Invalid request type"
        close(request.client_socket)
    END WHILE
```

Algorithm 4: Processing a resource request

```
process_resource_request(request):
    IF resource is not 1..6 THEN send error and RETURN
    IF duration <= 0 THEN send error and RETURN
    lock(resource_mutex)
    IF student already holds this resource THEN
        unlock(resource_mutex); send error; RETURN
    unlock(resource_mutex)

    sem = semaphore of the resource
    log "student waiting for resource"
    sem_wait(sem) // blocks if no instance is free

    lock(resource_mutex)
    available[resource] = available[resource] - 1
    unlock(resource_mutex)
    add_allocation(student, resource, duration) // start and end time
    log "student acquired resource"
    send "RESOURCE ALLOCATED" with details
```

6.2.4 Timer Thread

Algorithm 5: Automatic release

```
timer_thread():
    WHILE server_running DO
        sleep(1 second)
        now = current time
        lock(resource_mutex)
        FOR each entry in allocation table DO
            IF entry.allocated AND now >= entry.expected_end_time THEN
                entry.allocated = 0
                available[entry.resource] = available[entry.resource] + 1
                log "student released resource"
                sem_post(semaphore of entry.resource)
            END IF
        END FOR
        unlock(resource_mutex)
    END WHILE
```

6.3 Core C / C++ Implementation

The listing below shows the header files, constants, structures and global variables of the server. The constants make the configuration clear: the server listens on port

8080, the queue can hold 100 requests and there are three workers.

Listing 6.1: Declarations in lab_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <signal.h>
#include <arpa/inet.h>
#include <time.h>
#include <errno.h>

#define PORT 8080
#define MAX_CLIENTS 100
#define QUEUE_SIZE 100
#define WORKER_COUNT 3
#define BUFFER_SIZE 2048

/* Resource IDs */
#define COMPUTER 1
#define PRINTER 2
#define FILESERVER 3
#define PROJECTOR 4
#define SCANNER 5
#define HEADSET 6

typedef struct {
    int client_socket;
    int student_id;
    int request_type;
    int resource;
    int duration;
    char message[256];
} Request;

typedef struct {
    int student_id;
    int resource;
    int duration;
    time_t start_time;
    time_t expected_end_time;
    int allocated;
} Allocation;

/* Shared request queue */
Request request_queue[QUEUE_SIZE];
int queue_front = 0;
int queue_rear = 0;
int queue_count = 0;

pthread_mutex_t queue_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t queue_not_empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t queue_not_full = PTHREAD_COND_INITIALIZER;
pthread_mutex_t resource_mutex = PTHREAD_MUTEX_INITIALIZER;

/* Index 0 is not used */
int resource_total[7] = { 0, 10, 5, 5, 5, 5, 5 };
int resource_available[7] = { 0, 10, 5, 5, 5, 5, 5 };
```

```
sem_t computer_sem, printer_sem, fileserver_sem;
sem_t projector_sem, scanner_sem, headset_sem;

Allocation allocations[MAX_CLIENTS];
int allocation_count = 0;

volatile sig_atomic_t server_running = 1;
int server_socket = -1;
```

The variable `server_running` is declared as `volatile sig_atomic_t`. This type is the one that can be safely changed inside a signal handler and read by the other threads. All the loops of the server check this variable, so setting it to zero is enough to ask every thread to stop.

The next listing shows how the server socket is created and how the main thread accepts and parses the requests.

Listing 6.2: Socket creation and the accept loop in `main()`

```
server_socket = socket(AF_INET, SOCK_STREAM, 0);
if (server_socket < 0) {
    perror("Socket creation failed");
    server_running = 0;
    return 1;
}
setsockopt(server_socket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

if (bind(server_socket, (struct sockaddr *)&server_addr,
        sizeof(server_addr)) < 0) {
    perror("Bind failed");
    close(server_socket);
    return 1;
}
printf("[SERVER] Socket created successfully.\n");
printf("[SERVER] Server bound to port %d.\n", PORT);

if (listen(server_socket, MAX_CLIENTS) < 0) {
    perror("Listen failed");
    close(server_socket);
    return 1;
}
printf("[SERVER] Server is listening on port %d.\n", PORT);

while (server_running) {
    struct sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);

    int client_socket = accept(server_socket,
                              (struct sockaddr *)&client_addr,
                              &client_len);

    if (client_socket < 0) {
        if (!server_running) break;
    }
}
```

```

        if (errno == EINTR) continue;
        perror("Accept failed");
        continue;
    }

    char buffer[BUFFER_SIZE];
    memset(buffer, 0, sizeof(buffer));
    int bytes = recv(client_socket, buffer, sizeof(buffer) - 1, 0);
    if (bytes <= 0) { close(client_socket); continue; }
    buffer[bytes] = '\0';

    /* Format: student_id|request_type|resource|duration|message */
    Request request;
    memset(&request, 0, sizeof(request));
    request.client_socket = client_socket;

    char *token = strtok(buffer, "|");
    if (token != NULL) request.student_id = atoi(token);
    token = strtok(NULL, "|");
    if (token != NULL) request.request_type = atoi(token);
    token = strtok(NULL, "|");
    if (token != NULL) request.resource = atoi(token);
    token = strtok(NULL, "|");
    if (token != NULL) request.duration = atoi(token);
    token = strtok(NULL, "|");
    if (token != NULL)
        strncpy(request.message, token, sizeof(request.message) - 1);

    printf("\n[SERVER] Request received from Student %d.\n",
        request.student_id);
    add_request(request);
    printf("[SERVER] Request queued for Student %d.\n",
        request.student_id);
}

```

When `accept()` is interrupted by a signal it returns -1 with `errno` set to `EINTR`. The loop checks this case and simply continues, so sending `SIGUSR1` or `SIGUSR2` to the server does not disturb it. When the shutdown signal arrives, the handler closes the listening socket, `accept()` fails, and since `server_running` is now zero the loop ends.

The logging function is small but it is called from almost every other function.

Listing 6.3: Logging with Indian Standard Time

```

void get_indian_time(time_t timestamp, char *output, size_t size)
{
    struct tm *time_info = localtime(&timestamp);
    if (time_info != NULL)
        strftime(output, size, "%H:%M %d-%m-%Y", time_info);
    else
        strcpy(output, "Unknown");
}

void write_log(const char *message)
{
    FILE *file = fopen("server.log", "a");

```

```

    if (file == NULL)
        return;

    char time_string[64];
    get_indian_time(time(NULL), time_string, sizeof(time_string));
    fprintf(file, "[%s IST] %s\n", time_string, message);
    fclose(file);
}

```

At the beginning of `main()` the time zone is fixed with `setenv("TZ", "Asia/Kolkata", 1)` followed by `tzset()`. Because of this the function `localtime()` always gives Indian time, whatever the time zone setting of the machine is.

6.4 Linux System Calls / APIs Used

The table below lists every system call and library function used in the server together with the place where it is called.

Table 6.2: System calls and APIs in `lab_server.c`

Call	Where it is used	What it does
<code>socket()</code>	<code>main()</code>	Creates the listening TCP socket
<code>setsockopt()</code>	<code>main()</code>	Allows the port to be reused after a restart
<code>bind()</code>	<code>main()</code>	Attaches the socket to port 8080
<code>listen()</code>	<code>main()</code>	Marks the socket as ready to accept connections
<code>accept()</code>	<code>main()</code> loop	Takes the next waiting client connection
<code>recv()</code>	<code>main()</code> loop	Reads the request sent by the client
<code>send()</code>	worker functions	Sends the reply to the client
<code>close()</code>	<code>worker_thread()</code>	Closes the client connection after replying
<code>shutdown()</code>	<code>handle_signal()</code>	Stops the listening socket during shutdown
<code>signal()</code>	<code>main()</code>	Registers <code>handle_signal()</code> for four signals
<code>pthread_create()</code>	<code>main()</code>	Starts the worker threads and the timer thread
<code>pthread_join()</code>	<code>main()</code>	Waits for the threads to finish at shutdown

sem_init(), sem_destroy()	main()	Create and remove the six semaphores
sem_wait()	process_resource_request()	Acquires one instance of a resource
sem_post()	check_expired_allocations()	Returns one instance of a resource
sleep()	timer_thread()	Waits one second between checks
time(), difftime()	allocation and report functions	Start time, end time, used and remaining time
fopen(), fprintf(), fclose()	write_log()	Appends an entry to server.log
malloc(), free()	main(), worker_thread()	Memory for worker IDs and web page data

The client uses a smaller set of calls: `socket()` to create its end point, `inet_pton()` to convert the address 127.0.0.1 into binary form, `connect()` to reach the server, `send()` to send the request, `recv()` to read the reply and `close()` to end the connection.

Listing 6.4: Connecting and sending the request in lab_client.c

```

sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0) {
    perror("Socket creation failed");
    return 1;
}

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);

if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0) {
    perror("Invalid server address");
    close(sock);
    return 1;
}

if (connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0) {
    perror("Connection to server failed");
    close(sock);
    return 1;
}

snprintf(buffer, sizeof(buffer), "%d|%d|%d|%d|%s",
         student_id, request_type, resource, duration, message);

if (send(sock, buffer, strlen(buffer), 0) < 0) {
    perror("Failed to send request");
    close(sock);
    return 1;
}

memset(buffer, 0, sizeof(buffer));

```

```

int bytes_received = recv(sock, buffer, sizeof(buffer) - 1, 0);
buffer[bytes_received] = '\0';

printf("\n=====\\n");
printf("          SERVER RESPONSE\\n");
printf("=====\\n");
printf("%s\\n", buffer);
close(sock);

```

6.5 Process / Thread Implementation

The server is one process with five threads. The main thread exists from the beginning. The three worker threads and the timer thread are created in `main()` before the socket is opened, so that the workers are ready by the time the first request arrives.

Table 6.3: Threads of the server

Thread	Number	Work
Main thread	1	Accepts client connections and adds requests to the queue
Worker threads	3	Take requests from the queue and process them
Timer thread	1	Checks the usage time and releases the resource automatically

Listing 6.5: Creating and joining the threads

```

pthread_t workers[WORKER_COUNT];
pthread_t timer;

for (int i = 0; i < WORKER_COUNT; i++) {
    int *worker_id = malloc(sizeof(int));
    if (worker_id == NULL) {
        perror("Memory allocation failed");
        return 1;
    }
    *worker_id = i + 1;
    pthread_create(&workers[i], NULL, worker_thread, worker_id);
}

pthread_create(&timer, NULL, timer_thread, NULL);

/* ... server runs ... */

/* Shutdown */
server_running = 0;
pthread_cond_broadcast(&queue_not_empty);
pthread_cond_broadcast(&queue_not_full);

for (int i = 0; i < WORKER_COUNT; i++)
    pthread_join(workers[i], NULL);
pthread_join(timer, NULL);

```

Each worker needs to know its own number so that it can print messages such as

[WORKER 2] Processing Student 103 request. The number is passed through the last argument of `pthread_create()`. A separate integer is allocated with `malloc()` for every worker. If the address of the loop variable `i` were passed instead, all the workers might read the same changed value, which is itself a race condition.

Listing 6.6: The worker thread function

```
void *worker_thread(void *arg)
{
    int worker_id = *((int *)arg);
    free(arg);

    printf("[WORKER %d] Started.\n", worker_id);

    while (server_running) {
        Request request = get_request();

        if (!server_running && request.client_socket == 0)
            break;
        if (request.client_socket == 0)
            continue;

        printf("[WORKER %d] Processing Student %d request.\n",
               worker_id, request.student_id);

        if (request.request_type >= 1 && request.request_type <= 6)
            process_resource_request(request);
        else if (request.request_type == 7)
            show_resource_status(request.client_socket);
        else if (request.request_type == 8)
            show_all_allocations(request.client_socket);
        else if (request.request_type == 9)
            show_student_status(request.client_socket,
                                request.student_id);
        else {
            const char *message = "ERROR: Invalid request type.\n";
            send(request.client_socket, message, strlen(message), 0);
        }
        close(request.client_socket);
    }

    printf("[WORKER %d] Stopped.\n", worker_id);
    return NULL;
}
```

Listing 6.7: The timer thread function

```
void *timer_thread(void *arg)
{
    (void)arg;
    while (server_running) {
        sleep(1);
        check_expired_allocations();
    }
    return NULL;
}
```

The threads of the running server can be seen from another terminal with the

command `ps -T -p` followed by the process ID. The output shows five lines with the same process ID and five different thread IDs, which confirms that the server is one process with five threads.

6.6 IPC / Synchronization Implementation

6.6.1 Inter-Process Communication

The client process and the server process communicate through a TCP socket, as shown in Listings 6.2 and 6.4. Each request uses a new connection. The client connects, sends one line of text, receives the reply and closes the connection. Since TCP is reliable and keeps the order of bytes, no extra checking is required in our program.

6.6.2 Mutex and Condition Variables in the Request Queue

The request queue is the first shared data structure. The main thread writes to it and three workers read from it. The following listing shows how the mutex and the condition variables are used.

Listing 6.8: Thread-safe request queue

```
void add_request(Request request)
{
    pthread_mutex_lock(&queue_mutex);

    while (queue_count == QUEUE_SIZE && server_running)
        pthread_cond_wait(&queue_not_full, &queue_mutex);

    if (!server_running) {
        pthread_mutex_unlock(&queue_mutex);
        return;
    }

    request_queue[queue_rear] = request;
    queue_rear = (queue_rear + 1) % QUEUE_SIZE;
    queue_count++;

    pthread_cond_signal(&queue_not_empty);
    pthread_mutex_unlock(&queue_mutex);
}

Request get_request()
{
    Request request;
    memset(&request, 0, sizeof(Request));

    pthread_mutex_lock(&queue_mutex);

    while (queue_count == 0 && server_running)
```

```

        pthread_cond_wait(&queue_not_empty, &queue_mutex);

    if (!server_running && queue_count == 0) {
        pthread_mutex_unlock(&queue_mutex);
        return request;
    }

    request = request_queue[queue_front];
    queue_front = (queue_front + 1) % QUEUE_SIZE;
    queue_count--;

    pthread_cond_signal(&queue_not_full);
    pthread_mutex_unlock(&queue_mutex);
    return request;
}

```

Two points are important in this code. First, `pthread_cond_wait()` is called inside a while loop and not an if statement. When a thread wakes up, some other thread may already have taken the request, so the condition must be checked again. Second, `pthread_cond_wait()` releases the mutex while the thread sleeps and locks it again before returning. This allows the main thread to add a request while the workers are sleeping.

6.6.3 Semaphores for Limited Resources

The six semaphores are initialised in `main()` with the number of instances of each resource.

Listing 6.9: Initialising the semaphores

```

sem_init(&computer_sem, 0, 10);
sem_init(&printer_sem, 0, 5);
sem_init(&fileserversem, 0, 5);
sem_init(&projector_sem, 0, 5);
sem_init(&scanner_sem, 0, 5);
sem_init(&headset_sem, 0, 5);

```

The second argument 0 means that the semaphore is shared between the threads of one process. The third argument is the starting value. The core function that uses the semaphores is given below.

Listing 6.10: Allocating a resource

```

void process_resource_request(Request request)
{
    int resource = request.resource;

    if (resource < COMPUTER || resource > HEADSET) {
        const char *message = "ERROR: Invalid resource.\n";
        send(request.client_socket, message, strlen(message), 0);
        return;
    }
}

```

```

if (request.duration <= 0) {
    const char *message =
        "ERROR: Duration must be greater than 0 seconds.\n";
    send(request.client_socket, message, strlen(message), 0);
    return;
}

/* Duplicate allocation check */
pthread_mutex_lock(&resource_mutex);
if (has_duplicate_allocation(request.student_id, resource)) {
    pthread_mutex_unlock(&resource_mutex);
    char response[500];
    snprintf(response, sizeof(response),
        "ERROR: Student %d already has a %s allocated.\n",
        request.student_id, get_resource_name(resource));
    send(request.client_socket, response, strlen(response), 0);
    write_log(response);
    return;
}
pthread_mutex_unlock(&resource_mutex);

sem_t *sem = get_resource_semaphore(resource);

printf("[RESOURCE] Student %d waiting for %s.\n",
    request.student_id, get_resource_name(resource));

/* Semaphore controls limited resources */
sem_wait(sem);

pthread_mutex_lock(&resource_mutex);
resource_available[resource]--;
pthread_mutex_unlock(&resource_mutex);

add_allocation(request.student_id, resource, request.duration);

printf("\n[RESOURCE] %s acquired.\n", get_resource_name(resource));
printf("[RESOURCE] Student ID : %d\n", request.student_id);
printf("[RESOURCE] Duration      : %d seconds\n", request.duration);

/* ... build the "RESOURCE ALLOCATED" reply and send it ... */
}

```

The order of operations in this function is chosen carefully. The mutex is unlocked before `sem_wait()` is called. If the worker kept the mutex and then went to sleep on the semaphore, the timer thread would never be able to lock the mutex to release a resource, and the two threads would wait for each other for ever. By releasing the mutex first, this deadlock is avoided.

Listing 6.11: Recording and releasing an allocation

```

void add_allocation(int student_id, int resource, int duration)
{
    pthread_mutex_lock(&resource_mutex);
    if (allocation_count < MAX_CLIENTS) {
        Allocation *a = &allocations[allocation_count];
        a->student_id    = student_id;
        a->resource      = resource;
    }
}

```

```

        a->duration          = duration;
        a->start_time        = time(NULL);
        a->expected_end_time = a->start_time + duration;
        a->allocated         = 1;
        allocation_count++;
    }
    pthread_mutex_unlock(&resource_mutex);
}

void check_expired_allocations()
{
    time_t now = time(NULL);

    pthread_mutex_lock(&resource_mutex);
    for (int i = 0; i < allocation_count; i++) {
        if (allocations[i].allocated &&
            now >= allocations[i].expected_end_time) {

            int resource    = allocations[i].resource;
            int student_id = allocations[i].student_id;

            printf("[RESOURCE] Automatically releasing %s "
                   "from Student %d.\n",
                   get_resource_name(resource), student_id);

            release_allocation(i);          /* available++, allocated = 0 */

            sem_t *sem = get_resource_semaphore(resource);
            if (sem != NULL)
                sem_post(sem);
        }
    }
    pthread_mutex_unlock(&resource_mutex);
}

```

`sem_post()` never blocks, so it is safe to call it while holding the mutex. As soon as it is called, a worker that was sleeping in `sem_wait()` for the same resource becomes ready to run and gives the resource to the next student.

6.6.4 Signals

Signals are the communication channel between the administrator and the server. The handlers are registered in `main()` with `signal(SIGUSR1, handle_signal)` and similar calls for `SIGUSR2`, `SIGINT` and `SIGTERM`.

Listing 6.12: The signal handler

```

void handle_signal(int signal_number)
{
    if (signal_number == SIGUSR1) {
        pthread_mutex_lock(&queue_mutex);
        int pending = queue_count;
        pthread_mutex_unlock(&queue_mutex);

        printf("[SIGNAL] Pending requests: %d\n", pending);
    }
}

```

```

else if (signal_number == SIGUSR2) {
    printf("[SIGNAL] Resource status requested.\n");

    pthread_mutex_lock(&resource_mutex);
    printf("Computers      : %d/%d available\n",
           resource_available[COMPUTER], resource_total[COMPUTER]);
    printf("Printers       : %d/%d available\n",
           resource_available[PRINTER], resource_total[PRINTER]);
    /* ... same for the other four resources ... */
    pthread_mutex_unlock(&resource_mutex);

    write_log("SIGUSR2: Resource status requested.");
}
else if (signal_number == SIGINT || signal_number == SIGTERM) {
    printf("[SIGNAL] Shutdown signal received.\n");
    write_log("Shutdown signal received.");

    server_running = 0;
    pthread_cond_broadcast(&queue_not_empty);
    pthread_cond_broadcast(&queue_not_full);

    if (server_socket != -1) {
        shutdown(server_socket, SHUT_RDWR);
        close(server_socket);
        server_socket = -1;
    }
}
}

```

Table 6.4: Signals handled by the server

Signal	How it is sent	Action of the server
SIGUSR1	kill -SIGUSR1 <pid>	Shows pending requests
SIGUSR2	kill -SIGUSR2 <pid>	Shows resource status
SIGINT	Ctrl+C in the server terminal	Controlled shutdown of the server
SIGTERM	kill -SIGTERM <pid>	Controlled shutdown of the server

The shutdown is called controlled because the server does not simply die. The handler sets `server_running` to zero, wakes up every thread that is sleeping on a condition variable and closes the listening socket. The main thread comes out of its loop, joins the workers and the timer, destroys the semaphores, writes a final entry in the log and prints `[SERVER] Server stopped safely`.

6.7 Memory Management Implementation

The server uses three kinds of memory: static memory for the long-lived shared data, stack memory for the temporary data of each thread, and heap memory for data whose size or lifetime is decided at run time.

Table 6.5: Use of memory in the server

Kind	Data	Reason
Static (global)	request_queue[100], allocations[100], resource_total[], resource_available[], mutexes, semaphores	Must exist for the whole life of the server and be visible to all threads
Stack (local)	buffer[2048], response[], log_message[], Request copies	Needed only inside one function call; every thread has its own stack, so no locking is needed
Heap (dynamic)	worker_id for each worker, file_data for web pages	Lifetime or size is known only at run time

6.7.1 Static Allocation

The request queue and the allocation table are fixed-size global arrays. A fixed size has two advantages in a server. The memory is reserved once when the process starts, so there is no delay and no possibility of failure later. And the upper limit on memory is known in advance. The queue is circular, so the same 100 slots are used again and again and the queue never grows.

6.7.2 Stack Allocation

Every thread has its own stack. The buffers that a worker uses to build a reply are local variables, so they are on the stack of that worker. Two workers can therefore prepare two replies at the same time without disturbing each other and without any mutex. The stack memory is released automatically when the function returns.

6.7.3 Dynamic Allocation

Dynamic memory is used in two places. The first is the worker number passed to each worker thread (Listing 6.5). The memory is allocated in main() and freed by the worker itself as soon as it has copied the number (Listing 6.6). The second place is the code that sends the web pages. The size of a file is not known in advance, so the server finds the size, allocates exactly that much memory, reads the file into it, sends it and frees it.

Listing 6.13: Dynamic memory for sending a web page

```
fseek(frontend_file, 0, SEEK_END);
long file_size = ftell(frontend_file);
rewind(frontend_file);
```

```

char *file_data = malloc(file_size);
if (file_data == NULL) {
    fclose(frontend_file);
    close(client_socket);
    continue;
}

fread(file_data, 1, file_size, frontend_file);
fclose(frontend_file);

/* ... send the HTTP header and file_data ... */

free(file_data);

```

Care is taken to avoid the common memory errors. Every `malloc()` is checked for `NULL` and has a matching `free()`. All strings are written with `snprintf()` and `strncpy()`, which never write beyond the given size, so buffer overflow is avoided. Buffers are cleared with `memset()` before use, and the received text is always terminated with the null character.

6.8 Optional User Interface Implementation

Two user interfaces are provided: a terminal client and a web dashboard. Both are optional in the sense that the server logic does not depend on them, but they make the project easy to use and to demonstrate.

6.8.1 Terminal Client

The terminal client `lab_client.c` displays the menu given in Section 5.5, reads the student ID, the choice and the duration, and sends one request. Each run of the client is one student request. To show concurrency, several terminals are opened and the client is run in each of them with different student IDs.

6.8.2 Web Dashboard

The web dashboard is made with HTML, CSS and JavaScript and is served by the same C server at `http://localhost:8080`. When a connection arrives, the main thread looks at the first characters of the received text. If the text begins with `GET`, it is an HTTP request from a browser. The server then finds which file is asked for, reads that file from the `lab_frontend` folder and sends it with a proper HTTP header.

Listing 6.14: Serving the dashboard files from the C server

```
if (strncmp(buffer, "GET /", 5) == 0) {
    char file_path[256];
    char content_type[64];

    if (strncmp(buffer, "GET / ", 6) == 0 ||
        strncmp(buffer, "GET /index.html", 15) == 0) {
        strcpy(file_path, "lab_frontend/index.html");
        strcpy(content_type, "text/html");
    }
    else if (strncmp(buffer, "GET /style.css", 14) == 0) {
        strcpy(file_path, "lab_frontend/style.css");
        strcpy(content_type, "text/css");
    }
    else if (strncmp(buffer, "GET /script.js", 14) == 0) {
        strcpy(file_path, "lab_frontend/script.js");
        strcpy(content_type, "application/javascript");
    }
    else {
        const char *response =
            "HTTP/1.1 404 Not Found\r\n"
            "Content-Type: text/plain\r\n"
            "Content-Length: 9\r\n"
            "Connection: close\r\n"
            "\r\n"
            "Not Found";
        send(client_socket, response, strlen(response), 0);
        close(client_socket);
        continue;
    }

    FILE *frontend_file = fopen(file_path, "rb");
    /* ... read the file into file_data (Listing 6.13) ... */

    char header[512];
    snprintf(header, sizeof(header),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: %s\r\n"
        "Content-Length: %ld\r\n"
        "Connection: close\r\n"
        "\r\n",
        content_type, file_size);

    send(client_socket, header, strlen(header), 0);
    send(client_socket, file_data, file_size, 0);
    free(file_data);
    close(client_socket);
    continue;
}
```

The page itself does not contain any resource data. After it is loaded, the JavaScript code asks the C server for the data. For example, the dashboard page calls the address `/api/dashboard`, and the request form calls `/api/request` with the student ID, resource number and duration. The server answers these calls with the same text reports that the terminal client receives, and the JavaScript code reads the numbers

from the text and updates the cards on the page.

Listing 6.15: Sending a resource request from the browser (script.js)

```
async function serverRequest(url) {
  try {
    const response = await fetch(url, { cache: "no-store" });
    const text = await response.text();
    if (!response.ok) {
      throw new Error(text || "Server request failed.");
    }
    return text;
  }
  catch (error) {
    showResponse("SERVER CONNECTION ERROR\n\n" + error.message);
    updateAlert(
      "Unable to communicate with the laboratory server.",
      "error");
    return null;
  }
}

async function requestResource() {
  const studentId =
    document.getElementById("studentId").value.trim();
  const resource =
    Number(document.getElementById("resource").value);
  const duration =
    Number(document.getElementById("duration").value);

  if (!studentId) {
    alert("Please enter Student ID.");
    return;
  }
  if (!duration || duration <= 0) {
    alert("Please enter a valid usage time.");
    return;
  }

  const url = "/api/request" +
    "?student_id=" + encodeURIComponent(studentId) +
    "&resource=" + encodeURIComponent(resource) +
    "&duration=" + encodeURIComponent(duration);

  showPageById("response");
  const response = await serverRequest(url);
  if (response === null) return;
  showResponse(response);
}

async function loadDashboard() {
  const response = await serverRequest("/api/dashboard");
  if (response === null) return;
  updateResourceCounts(response);
  updateDashboardAlert(response);
}
```

Table 6.6: Pages of the web dashboard

Page	What it shows
Dashboard	shows available resources

Request Resource	student ID, resource and usage time
Student Status	resource used by a student
Allocated Resources	all current allocations
Server Response	the exact reply text received from the C server

Because the browser talks to the same lab_server process, a resource requested from the web page reduces the same counter and the same semaphore as a resource requested from the terminal client. If a computer is requested from the terminal, the count on the web dashboard also changes after a refresh. This shows that there is only one source of truth, which is the C server.

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
Enter Student ID: 101

----- MENU -----
1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status
-----

Enter your choice: 1

Enter required usage time (in seconds): 10

=====
                SERVER RESPONSE
=====

=====
                RESOURCE ALLOCATED
=====
Student ID    : 101
Resource      : Computer
Duration      : 10 seconds
Start Time    : 20:17 23-09-2026 IST
Expected End  : 20:17 23-09-2026 IST
-----
The resource will be automatically
released after 10 seconds.
=====
```

```
=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====
```

Enter Student ID: 101

```
----- MENU -----
```

1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status

```
-----
```

Enter your choice: 4

```
=====
SERVER RESPONSE
=====
```

```
=====
LAB RESOURCE DASHBOARD
=====
```

```
Computers      : 4/5 available
Printers       : 2/2 available
File Servers   : 2/2 available
```

```
=====
```

7.2 Execution Commands

Terminal-1

```
sushma_sree@LAPTOP-6PJSRT08:~$ cd project
sushma_sree@LAPTOP-6PJSRT08:~/project$ ls
Makefile  client.c  lab_client  lab_server  server.c  thread_test
a.out     common.h  lab_client.c  lab_server.c  server.log  thread_test.c
```

Compile the server:

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ gcc lab_server.c -o lab_server -pthread
sushma_sree@LAPTOP-6PJSRT08:~/project$ |
```

Run the server:

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_server
[WORKER 1] Started.
[WORKER 2] Started.
[WORKER 3] Started.
[SERVER] Socket created successfully.
[SERVER] Server bound to port 8080.
[SERVER] Server is listening on port 8080.
[SERVER] Indian Time: 20:14 23-09-2026 IST
```

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_client

=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====

Enter Student ID: |
```

Terminal-2

Compile the client:

```
sushma_sree@LAPTOP-6PJSRT08:~$ cd project
sushma_sree@LAPTOP-6PJSRT08:~/project$ gcc lab_client.c -o lab_client
sushma_sree@LAPTOP-6PJSRT08:~/project$ |
```

Run the client:

7.3 Sample Inputs and Outputs

**LABCORE**
University Laboratory Management System

SERVER ONLINE

DashboardRequest ResourceStudent StatusAllocated ResourcesServer Response

RESOURCE ALLOCATION

Request Laboratory Resource

Submit a request to allocate a laboratory resource.

New Resource Request

The request will be processed by the laboratory management server.

Student ID

Resource

Usage Time

101

Computer

100

Resource will be automatically released after the requested duration.

Request Resource →

**LABCORE**
University Laboratory Management System

SERVER ONLINE

DashboardRequest ResourceStudent StatusAllocated ResourcesServer Response

Server Response

View the latest response received from the C laboratory server.

Latest Server Message

Communication between the web interface and C server.

```
=====
RESOURCE ALLOCATED
=====
Student ID   : 101
Resource     : Computer
Duration     : 100 seconds
Start Time   : 10:54 08-10-2026 IST
Expected End : 10:56 08-10-2026 IST
=====
The resource will be automatically
released after 100 seconds.
=====
```

7.4 Screenshots of Execution

LABCORE
University Laboratory Management System

SERVER ONLINE

Dashboard Request Resource Student Status Allocated Resources Server Response

LABORATORY CONTROL CENTER

Laboratory Dashboard

Monitor laboratory resources and server availability from one centralized interface.

Refresh

System Status
All laboratory resources are being monitored normally.

LIVE RESOURCE STATUS
Resource Availability LIVE

Resource Type	Available Resources
COMPUTER	9
PRINTER	5
FILE SERVER	5

LABCORE
University Laboratory Management System

SERVER ONLINE

Dashboard Request Resource Student Status Allocated Resources Server Response

Resource Type	Available Resources
PROJECTOR	5
SCANNER	5
HEADSET	5

QUICK ACCESS
Laboratory Operations

Operation	Action
Request Resource	Allocate a laboratory resource
Student Status	Check current allocation
Allocations	View active resources
Server Response	View latest server message

7.5 Demonstration of OS Concepts

Request queue

```

sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_client

=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====

Enter Student ID: 101

----- MENU -----
1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status
-----

Enter your choice: 2

Enter required usage time (in seconds): 10

=====
SERVER RESPONSE
=====

=====
RESOURCE ALLOCATED
=====
Student ID    : 101
Resource      : Printer
Duration      : 10 seconds
Start Time    : 20:26 23-09-2026 IST
Expected End  : 20:27 23-09-2026 IST
-----
The resource will be automatically
released after 10 seconds.
=====

```

Semaphore

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_client
```

```
=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====

Enter Student ID: 102

----- MENU -----
1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status
-----

Enter your choice: 2

Enter required usage time (in seconds): 20

=====
SERVER RESPONSE
=====

=====
RESOURCE ALLOCATED
=====
Student ID    : 102
Resource      : Printer
Duration      : 20 seconds
Start Time    : 20:28 23-09-2026 IST
Expected End  : 20:28 23-09-2026 IST
-----
The resource will be automatically
released after 20 seconds.
=====
```

Allocate the second printer(semaphore)

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_client
```

```
=====
UNIVERSITY LAB MANAGEMENT CLIENT
=====
```

```
Enter Student ID: 103
```

```
----- MENU -----
```

1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status

```
-----
Enter your choice: 2
```

```
Enter required usage time (in seconds): 60
```

```
=====
SERVER RESPONSE
=====
```

```
=====
RESOURCE ALLOCATED
=====
```

```
Student ID    : 103
Resource      : Printer
Duration      : 60 seconds
Start Time    : 20:30 23-09-2026 IST
Expected End  : 20:31 23-09-2026 IST
```

```
-----
The resource will be automatically
released after 60 seconds.
=====
```

Request for 3printer

```
sushma_sree@LAPTOP-6PJSRT08:~/project$ ./lab_client

=====
      UNIVERSITY LAB MANAGEMENT CLIENT
=====

Enter Student ID: 104

----- MENU -----
1. Request Computer
2. Request Printer
3. Request File Server
4. View Resource Dashboard
5. View All Allocated Resources
6. View My Resource Status
-----

Enter your choice: 2

Enter required usage time (in seconds): 90
|
```

In server we get as,

```
[SERVER] Request received from Student 104.
[SERVER] Request queued for Student 104.
[WORKER 1] Processing Student 104 request.
[RESOURCE] Student 104 waiting for Printer.
```

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The University Laboratory Management Server is a multi-threaded server program written in C on Linux (Ubuntu). It manages the limited resources of a computer laboratory, namely computers, printers, file servers, projectors, scanners and headsets. Students send their requests from a terminal client or from a web dashboard. The server accepts many requests at the same time, keeps them in a shared circular queue and processes them with a pool of three worker threads.

Mutexes protect the request queue and the allocation table from race conditions. One counting semaphore for each resource type makes sure that the number of students using a resource never becomes more than the number of instances available. A timer thread releases every resource automatically when its usage time is over. The administrator can see the pending requests and the resource status at any time by sending signals, and can shut the server down in a controlled manner. All activities are stored with date and time in a log file.

In this way the project brings together the important topics of the operating systems course, namely processes, threads, synchronisation, inter-process communication, signals, file handling and memory management, in one complete and working application.

8.2 Objectives Achieved

Table 8.1: Objectives and how they were achieved

Objective	How it was achieved	Status
Accept multiple client requests simultaneously	TCP server socket on port 8080 with an accept loop in the main thread	Achieved
Use worker threads to process student requests	Three worker threads created with <code>pthread_create()</code>	Achieved
Maintain a shared request queue	Circular queue of 100 requests	Achieved
Use mutexes to protect shared data structures	<code>queue_mutex</code> and <code>resource_mutex</code>	Achieved

Use semaphores to control limited resources	Six counting semaphores, one for each resource type	Achieved
Maintain student requests and resource allocation information	Request structure and allocation table with start and end times	Achieved
Use signals for server administration and controlled shutdown	Handlers for SIGUSR1, SIGUSR2, SIGINT and SIGTERM	Achieved
Monitor threads and resource usage	Worker messages, dashboard, SIGUSR2 status and ps -T	Achieved
Maintain server activity logs	write_log() appends to server.log with IST time	Achieved

8.3 Results Achieved

- The server handles many student requests at the same time.
- Threads, mutexes and semaphores make resource sharing safe.
- Resources are released automatically after the usage time.
- Signals, logs and dashboard help to monitor the server.
- A student who asks for a resource that is fully in use is made to wait and gets the resource as soon as one instance is released.
- A student cannot get two instances of the same resource at the same time, because duplicate requests are detected and rejected.
- The terminal client and the web dashboard always show the same resource counts, since both are served by the same C server.
- The server stops safely on Ctrl+C: all threads are joined and the semaphores are destroyed before the process exits.

The following improvements can be taken up as future scope of the project:

- Student login with password.
- Storing the data in a database.
- Priority for urgent requests.

8.4 Skills and Learning Outcomes

Working on this project gave us practical experience that we could not have got only from the classroom. The main skills and learning outcomes are listed below.

- Systems programming in C: we learnt to write a complete program using structures, pointers, string handling and dynamic memory, and to compile it with GCC.
- Working in Linux: we became comfortable with the Ubuntu terminal, the nano editor and commands such as ps, kill, cat and curl, and with using several terminals together.
- Socket programming: we understood the steps socket, bind, listen and accept on the server side and connect on the client side, and how data is exchanged with send and recv.
- Multithreading: we learnt how to create and join threads, how to pass arguments to a thread and why a thread pool is better than creating a thread for every client.
- Synchronisation: we saw race conditions in practice and learnt how mutexes, condition variables and semaphores remove them. We also learnt how the wrong order of locking can lead to deadlock and how to avoid it.
- Signals: we learnt to write a signal handler and to use signals for communicating with a running process.
- File handling and logging: we learnt how a log file helps to understand what a concurrent program actually did.
- Web basics: by serving HTML, CSS and JavaScript from our own C program, we understood what a web server really does and how a browser talks to it through HTTP.
- Debugging: we learnt to read compiler errors, to test one step at a time and to find mistakes from the server messages and the log file.
- Team work: we divided the work among the team members, shared the code

through GitHub and prepared the presentation and the documentation together.

Above all, the project changed the way we look at an operating system. Concepts such as the critical section, the bounded buffer and the counting semaphore are no longer only definitions for us; they are parts of a program that we have written, run and tested ourselves.

CHAPTER -9 REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.
2. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA: Pearson, 2014.
3. W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
4. M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook. San Francisco, CA: No Starch Press, 2010.
5. W. R. Stevens, B. Fenner and A. M. Rudoff, UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Boston, MA: Addison-Wesley, 2003.
6. D. R. Butenhof, Programming with POSIX Threads. Reading, MA: Addison-Wesley, 1997.
7. B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

9.2 Research Papers

1. E. W. Dijkstra, "Cooperating sequential processes," Technical Report EWD-123, Technological University, Eindhoven, The Netherlands, 1965.
2. C. A. R. Hoare, "Monitors: An operating system structuring concept," Communications of the ACM, vol. 17, no. 10, pp. 549-557, Oct. 1974.
3. E. G. Coffman, M. J. Elphick and A. Shoshani, "System deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67-78, June 1971.
4. D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," Communications of the ACM, vol. 17, no. 7, pp. 365-375, July 1974.
5. A. D. Birrell, "An introduction to programming with threads," Research

Report 35, Digital Equipment Corporation Systems Research Center, Palo Alto, CA, 1989.

9.3 Linux / GNU Documentation

1. Linux man-pages project, pthreads(7), pthread_create(3), pthread_mutex_lock(3p), pthread_cond_wait(3p). Available: <https://man7.org/linux/man-pages/>
2. Linux man-pages project, sem_overview(7), sem_init(3), sem_wait(3), sem_post(3). Available: <https://man7.org/linux/man-pages/>
3. Linux man-pages project, socket(2), bind(2), listen(2), accept(2), connect(2), send(2), recv(2). Available: <https://man7.org/linux/man-pages/>
4. Linux man-pages project, signal(7), signal(2), kill(1). Available: <https://man7.org/linux/man-pages/>
5. Free Software Foundation, Using the GNU Compiler Collection (GCC). Available: <https://gcc.gnu.org/onlinedocs/>
6. Free Software Foundation, The GNU C Library Reference Manual. Available: <https://www.gnu.org/software/libc/manual/>
7. IEEE and The Open Group, The Open Group Base Specifications Issue 7 (POSIX.1-2017). Available: <https://pubs.opengroup.org/onlinepubs/9699919799/>

9.4 Online Resources

1. B. Hall, Beej's Guide to Network Programming: Using Internet Sockets. Available: <https://beej.us/guide/bgnet/>
2. Mozilla Developer Network, "Using the Fetch API." Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
3. Mozilla Developer Network, "An overview of HTTP." Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
4. GeeksforGeeks, "Multithreading in C" and "Socket Programming in C."

Available: <https://www.geeksforgeeks.org/>

5. Ubuntu Documentation. Available: <https://help.ubuntu.com/>
6. Microsoft, "Windows Subsystem for Linux Documentation." Available: <https://learn.microsoft.com/en-us/windows/wsl/>
7. GitHub Docs, "Get started with GitHub." Available: <https://docs.github.com/en/get-started>